Beyond Nominal Ratios: Language-Aware Prompt Compression for Multi-hop Question Answering

Thanh-Nhan Truong[1,2] [0009−0007−5012−7578]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
25C1505320@student.hcmus.edu.vn
[2] Vietnam National University, Ho Chi Minh City, Vietnam

Abstract. Prompt compression is reported as a ratio: the fraction of the context a compressor keeps. We show that this number is not comparable across languages, and that the gap is large enough to change which method looks cheaper. Measuring token fertility on parallel WMT24++ sentences across four writing systems, a context costs 4.97× more tokens in Hindi, 3.89× in Thai and 2.49× in Vietnamese than its English counterpart under the tokeniser that bills, so a compressor reporting the same nominal ratio in two languages is not delivering the same saving. We propose an *effective ratio* that prices retention in English-equivalent tokens, and show it separates tokenisers that nominal ratios rank as identical (1.02× spread nominal, 1.80× effective). To ground this we reimplement IterCOMP, a state-of-the-art training-free compressor evaluated only on English, and give it its first systematic evaluation on Vietnamese using the complete VimQA benchmark alongside MuSiQue, 2WikiMultiHopQA and HotpotQA ($n \geq 1{,}003$ each, six compression baselines). Two further effects appear only outside English: a boolean-answer normalisation step is worth +8.67 F1 in Vietnamese and exactly 0.0 in English, and undiacriticised text – ubiquitous online – costs every branch about 20 F1 while leaving the ordering intact, the damage falling on the reader rather than on the compressor. We release the implementation and the Vietnamese prompts.

Keywords: Prompt compression · Multi-hop question answering · Tokeniser fertility · Vietnamese NLP · Retrieval-augmented generation

1 Introduction

Multi-hop question answering requires aggregating evidence across documents. Retrieval-augmented generation (RAG) [7] supplies external documents to compensate for a model's bounded knowledge, but introduces two problems: *cost*

Corresponding author: Thanh-Nhan Truong, 25C1505320@student.hcmus.edu.vn. Code, prompts (Vietnamese and English), per-dataset results and the extended analyses are at https://github.com/nhantrnh/IterCOMP-X.

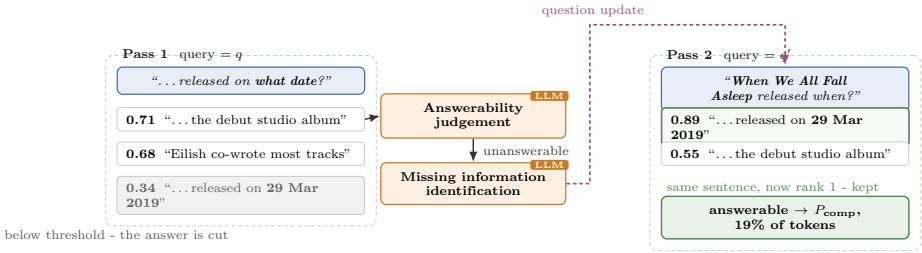


Fig. 1. One VimQA question through two IterCOMP passes. The sentence carrying the release date scores 0.34 and is filtered out, so a single-pass compressor loses the answer; re-scored against the follow-up q' it rises to 0.89 and is kept.

(input length grows linearly with the number of documents) and *noise* (irrelevant passages degrade reader accuracy).

Prompt compression addresses both. However, existing methods [4,10,15,8] compress *once*, conditioned on the original question, and therefore miss the defining property of multi-hop QA: the information needed at a later hop only becomes identifiable after the evidence of the earlier hop is in hand. IterCOMP [16] embeds multi-hop reasoning into the compression loop itself. No public implementation accompanies the original paper, and its evaluation remains restricted to English.

The Vietnamese adaptation and evaluation are our primary contribution, and the reproduction that enables them is reported after it. Five findings follow, four specific to Vietnamese:

- a boolean-answer normalisation step worth +8.67 F1 on Vietnamese (CI $[+6.98, +10.47]$) and exactly 0.0 on all three English datasets, a linguistic fix rather than a metric-inflating one;
- evidence that the iteration budget saturates beyond two passes, with `max_iter` from 2 to 5 moving F1 by +1.2 at $n = 500$ (within noise) while retention grows from 0.160 to 0.194;
- an *effective ratio* showing that nominal compression ratios understate real token cost on Vietnamese by 2.39×, since the tokeniser that bills is not the one that counts;
- that Vietnamese does not handicap the compressor, IterCOMP-X reaching 84% of the Oracle ceiling on VimQA, the lower 56% on English MuSiQue being a dataset property, not a language effect (Section 5.5);
- that undiacriticised Vietnamese costs compressed and gold context alike about 20 F1, leaving their ordering unchanged (Table 7).

We call this configuration IterCOMP-X and stress that it is *not* a new algorithm: the loop is unchanged. It is a deployment configuration plus a metric-level fix, and we report which parts survive significance testing.

None of this is measurable without a working reimplementation; absent a public one, we rebuild it from the paper's description and redesign every prompt. It recovers the central result on all four datasets (Table 5); Section 5.2 compares

our numbers with the original's, and the released repository carries the per-dataset breakdown.

2 Related Work

Prompt compression. Extractive methods keep a subset of tokens or sentences: LLMLingua [4] drops tokens by perplexity; LongLLMLingua [5] adds query awareness; LLMLingua-2 [10] recasts this as token classification, faster and *multilingual* - the decisive property here; Selective-Context [8] removes redundant lexical units. *Abstractive* methods regenerate shorter content, notably RECOMP [15]. All compress in a single pass.

Iterative retrieval. Self-Ask [13] and IRCoT [14] decompose questions and interleave retrieval with reasoning; IterCOMP instead *narrows* the context rather than expanding it. Every one of these is reported on English benchmarks.

Tokeniser disparity. Subword tokenisers split non-English text more finely, so the same content costs more tokens: Ahia et al. [1] price this against commercial APIs and Petrov et al. [11] frame it as unfairness between languages. That literature measures the cost of *text*. What it has not asked is what the disparity does to a *ratio* of two token counts, which is how every compressor above reports its result - the question Section 3.1 takes up.

Vietnamese NLP. The language is well served at the model level - PhoBERT [9] and ViT5 [12] are established encoders - and VimQA [6] gives it a multi-hop benchmark built to the HotpotQA format. What is missing is the layer between: to our knowledge no compressor has been evaluated on Vietnamese, so neither the accuracy nor the cost of compressing a Vietnamese context has been measured.

3 Methodology

3.1 The tokeniser tax and the effective ratio

A compression ratio is reported as a fraction of the context kept, and read as a saving. That reading assumes the unit being counted costs the same everywhere. It does not: the tokeniser that bills splits some writing systems far more finely than English, so the same nominal ratio buys different savings in different languages. We measure the gap here and propose a correction.

Four writing systems The claim above is about Vietnamese, so it needs a control: if every language were billed alike, fertility would not be worth reporting. Measuring 400 parallel WMT24++ pairs per language (`fertility_multilingual.py`) shows it is not.

Table 1. Token fertility over 930 parallel English-Vietnamese sentence pairs from WMT24++ [3]. All 95% confidence intervals are narrower than ±0.04. Fertility depends strongly on the tokeniser: a factor of two separates `cl100k_base` from XLM-R. [a]GPT-4, GPT-3.5-turbo; [b]GPT-4o. Which tokeniser bills depends on the served model, so exposure to fertility is set by that choice as much as by the language.

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base`[a]	105.6	44.1	**2.39**×
`o200k_base`[b]	63.3	43.3	1.49×
XLM-R (used by bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

Every language exceeds 1.0 with non-overlapping intervals (Fig. 2), so the mismatch is systematic rather than a Vietnamese quirk; Vietnamese at 2.49× is the *mild* case, Hindi being penalised twice as heavily. The 2.49× here (400 pairs) and the 2.39× in Table 1 (930 pairs) are different samples of the same corpus.

Crucially, fertility depends on the tokeniser measured. The compression ratio (Table 6) counts with `\w+`, where fertility is only 1.34×, so comparing ratios across languages is close to fair; but *API cost* is billed on the served model's tokeniser - 2.39× under `cl100k_base` against 1.49× under `o200k_base` - while bge-m3's XLM-R is only 1.19×. On the 200 VimQA contexts here a context averages 1,110 `\w+` words but 2,364 `cl100k_base` tokens, so our ratios *understate* billing 2×.

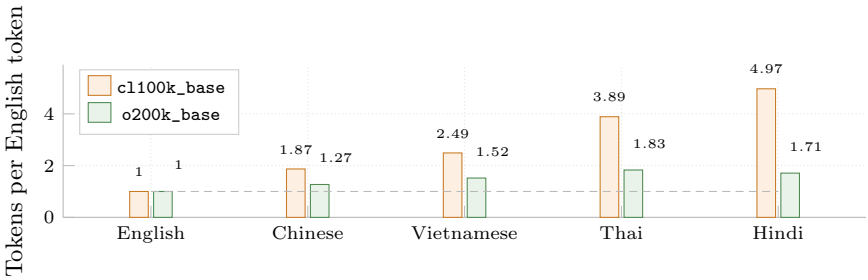


Fig. 2. What the same content costs in tokens, relative to its English translation, over 400 WMT24++ pairs per language. English is 1.0 by construction (dashed). Under `cl100k_base` a Hindi context costs almost five times its English counterpart, so two systems reporting the same nominal ratio in Hindi and in English are not delivering the same saving. Changing the tokeniser changes the tax more than changing the language does: `o200k_base` flattens every bar, and reorders Thai above Hindi.

The effective ratio Write $|x|_{\mathcal{T}}$ for the number of tokens the billed tokeniser $\mathcal{T}$ assigns to a string x, and let D be the retrieved documents and P_{comp} the compressed prompt. The quantity compression papers report is

$$\text{Ratio}_{\text{nom}} \;=\; \frac{|P_{\text{comp}}|_{\mathcal{T}}}{|D|_{\mathcal{T}}}. \tag{1}$$

Let $\phi_{\mathcal{T}}(\mathcal{L})$ be the fertility of language $\mathcal{L}$ under $\mathcal{T}$: the mean token count of a text in $\mathcal{L}$ divided by that of its English translation, which is what Fig. 2 measures. We define the *effective ratio* as what the compressed context costs against an English original of the same content,

$$\text{Ratio}_{\text{eff}} \;=\; \text{Ratio}_{\text{nom}} \cdot \phi_{\mathcal{T}}(\mathcal{L}), \tag{2}$$

so that (2) gives $\text{Ratio}_{\text{eff}} = \text{Ratio}_{\text{nom}}$ for English, where $\phi = 1$, and rises above the nominal figure exactly as far as the language is penalised by the tokeniser. Keeping a fifth of a Vietnamese context is not the saving that keeping a fifth of an English one is, and the second number says by how much. The correction is deliberately a single factor: fertility belongs to the language-tokeniser pair rather than to the compressor, so it is applied to the retention figure rather than charged against the compressor's score. Reporting the pair $(\text{Ratio}_{\text{nom}}, \text{Ratio}_{\text{eff}})$ costs one extra column and leaves the familiar number intact.

Does the effective ratio discriminate? A second metric that moved with the first would add nothing. Holding the compressed contexts fixed - the same 50 VimQA questions, the same IterCOMP output, only the tokeniser changed - the nominal ratio barely moves while the effective ratio does:

Table 2. Nominal against effective ratio for one fixed set of compressed contexts (50 VimQA questions, same IterCOMP-X output, tokeniser varied).

Tokeniser	nominal	effective
`cl100k_base`	0.236	**0.564**
`o200k_base`	0.240	0.357
`\w+`	0.234	**0.314**
spread	1.02×	**1.80**×

Because the contexts are byte-identical across the three rows, the whole of that 1.80× is the tokeniser's and none of it the compressor's. A nominal ratio reported on its own therefore conceals all of it, which is the case for adding the second column rather than replacing the first.

3.2 Iterative multi-hop compression

Given a question q and document set D, we seek a compressed context P_{comp} with $|P_{\text{comp}}| \ll |D|$ such that the reader still answers correctly. IterCOMP is

training-free: all parameters stay frozen. Each document is decomposed into sentence-level evidence segments $e_{i,j}$. Figure 1 traces one question through two iterations.

Relevant evidence filtering The *semantic* signal is the inner product of dense representations from a text encoder $E(\cdot)$:

$$S_{\text{sem}}(q, e_{i,j}) = E(q)^\top E(e_{i,j}). \tag{3}$$

This can miss exact lexical cues. Following M3-Embedding [2], the importance weight of token t is

$$w_t = \text{ReLU}(\mathbf{w}_{\text{lex}}^\top E(t)), \tag{4}$$

where $\mathbf{w}_{\text{lex}}$ is a projection vector. The *lexical* score is a sparse inner product over co-occurring tokens:

$$S_{\text{lex}}(q, e_{i,j}) = \sum_{t \in q \cap e_{i,j}} w_t^q \cdot w_t^{e_{i,j}}. \tag{5}$$

The two are combined into a dual-aspect relevance score:

$$S_{\text{dual}} = \lambda\, S_{\text{sem}} + (1 - \lambda)\, S_{\text{lex}}. \tag{6}$$

Filtering uses a percentile threshold rather than a fixed top-k:

$$E_{\text{cand}} = \{e_{i,j} \mid S_{\text{dual}} \geq \text{Percentile}(S, k)\}. \tag{7}$$

The threshold therefore *adapts* to each question's score distribution rather than committing to a fixed budget.

Answerability judgment and follow-up generation These two decision points are what make IterCOMP "reasoning-aware": the query evolves with the evidence state instead of staying fixed at q. In step 2 the reader is shown the evidence accumulated so far and asked whether it suffices to answer q, replying with a single token. The reply is read strictly: anything other than `ANSWERABLE_YES` continues the loop, so a malformed answer costs an extra pass rather than a wrong stop. When the evidence is judged insufficient, step 3 asks for one short follow-up question naming the information still missing, and that follow-up becomes q' for the next pass. The loop ends at whichever comes first, a sufficient verdict or `max_iter`. Neither prompt is published in the original paper; ours are released in full, in Vietnamese and English, with the code.

4 Experimental Setup

4.1 Datasets

The three English datasets are standard for multi-hop QA and are exactly those used by the original paper. VimQA [6] is a Vietnamese multi-hop QA dataset

Table 3. Development-set sizes. #Q is the full split, not the sample evaluated: the English sets are run at $n = 1{,}500$ and VimQA in full.

Dataset	Lang.	#Q	#psg
HotpotQA (distractor)	En	7,405	10
2WikiMultiHopQA	En	12,576	10
MuSiQue-Ans	En	2,417	20
VimQA	**Vi**	**1,003**	**10**

Table 4. Hyperparameters, following the original paper (which used a LLaMA-3-8B reader); VimQA, absent there, uses $k = 90$ as HotpotQA. The reader is quantised to 4-bit NF4 with double quantisation (bitsandbytes) and served through HuggingFace Transformers; decoding is greedy (`do_sample=False`) with at most 48 new tokens, so no run-to-run sampling variance enters the numbers.

Parameter	Value
Max iterations	5
λ	0.6
k (percentile)	90; 85 for 2Wiki
Encoder	bge-m3 (frozen)
Reader	Qwen2.5-7B-4bit

built from Vietnamese Wikipedia, annotated with reasoning type (bridge / comparison) and gold supporting sentences; crucially, it uses the same format as HotpotQA, so our code runs on Vietnamese essentially unchanged. We use the distractor setting for all four, giving 10-20 candidate passages per question, so no Wikipedia index is required and cost is per question rather than per corpus. Because MuSiQue withholds test answers, we report on development sets throughout, and all four datasets are evaluated under a single reader (Qwen2.5-7B at 4-bit) so the numbers compare like with like; Table 3 gives the statistics.

We adopt the original paper's hyperparameters (Table 4).

4.2 Baselines

Raw Document concatenates everything with no compression. Oracle keeps gold supporting sentences only - the upper bound for any document-level method, and a diagnostic: if Oracle is also low, the bottleneck is the reader rather than the compressor. LLMLingua-2 [10] is our main compression baseline, chosen because its token classifier is natively multilingual and covers Vietnamese. Against these we run IterCOMP as reimplemented here.

4.3 Metrics

We report Exact Match (EM) and F1 under the standard HotpotQA normalisation, plus the compression ratio (compressed over Raw Document tokens). That

Table 5. Main results: F1*, Qwen2.5-7B at 4-bit; the second row gives n. The last two rows are IterCOMP-X minus that baseline. † paired bootstrap interval excludes zero. Bold marks whichever of Raw and IterCOMP-X is higher. Per-dataset discussion is in the released repository.

	VimQA 1003	MuSiQue 1500	2Wiki 1500	HotpotQA 1500
Oracle	61.03	44.90	57.65	**71.40**
Raw	46.87	21.16	35.72	**59.24**
IterCOMP-X	**51.29**	**25.35**	**36.72**	54.58
LLML-2	35.31	14.63	19.24	36.00
Δ vs. Raw	$+4.42^{\dagger}$	$+4.19^{\dagger}$	$+1.00$	$\mathbf{-4.66}^{\dagger}$
Δ vs. LLML-2	$+15.98^{\dagger}$	$+10.72^{\dagger}$	$+17.48^{\dagger}$	$+18.58^{\dagger}$

normalisation is built for English, so for Vietnamese we preserve diacritics - stripping them collapses distinct words (*má* / *mà*) and renders EM/F1 meaningless. The released code documents this and the other text-handling prerequisites.

5 Results and Discussion

5.1 Main results across four datasets

The central claim holds on all four. Fig. 3 plots accuracy against context retained; sentence-level compression beats token-level LLMLingua-2 by +10.72 to +18.58 F1 (Table 5, last row), every paired bootstrap interval clear of zero. The two are not matched on cost: LLMLingua-2 classifies tokens once with a small encoder, IterCOMP-X calls a 7B model twice per pass over 3.1 passes, 3.2 to 7.2× the time per question (Section 5.3). The margin says sentence-level selection keeps multi-hop evidence that token-level scoring discards, not that it is cheaper. This is the paper's thesis, and the one result that does not depend on which dataset, reader or sample size we look at. It survives a wider field too: on VimQA ($n = 200$) IterCOMP-X leads all six baselines we added, but only LLMLingua-2 is multilingual and meets Vietnamese on equal terms, where the margin is +11.36 F1, CI $[+4.93, +17.84]$, $p = 0.0008$ against a 7.7 threshold. The other five carry an English-only component (GPT-2 scoring; NQ- and HotpotQA-trained RECOMP; English-fine-tuned CompAct) and appear in full in the released results.

The full ordering does not reproduce uniformly. The paper reports Oracle > IterCOMP > Raw > LLMLingua-2. We reproduce it on VimQA (IterCOMP-X−Raw +4.42, interval clear) and on MuSiQue (+4.19, interval clear; at $n = 500$ the same margin read +3.31 and crossed zero, so tripling the sample decided it); the order matches on 2Wiki, where +1.00 does not separate; and on HotpotQA

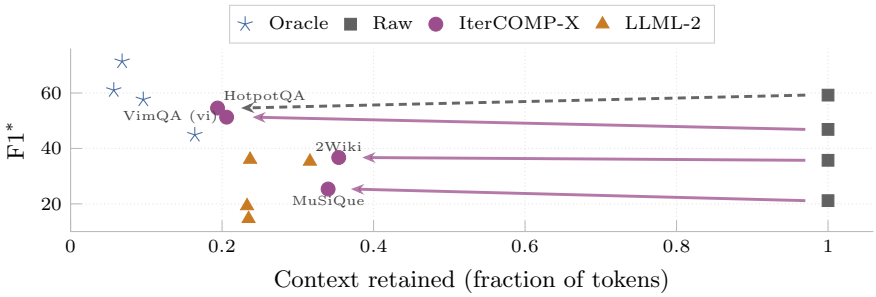


Fig. 3. Accuracy against context retained, all four datasets. Up and to the left is better. Each arrow runs from a dataset's Raw point to its IterCOMP-X point: on VimQA, MuSiQue and 2Wiki the arrow rises while moving left, so compression buys accuracy *and* tokens. On HotpotQA (dashed) it falls: the reader handles the full context well enough that filtering costs 4.66 F1, and only the token saving survives. Oracle marks the ceiling perfect selection would reach.

Raw overtakes IterCOMP-X by 4.66 F1, interval clear. The IterCOMP-X−Raw gap across the four is therefore +4.42, +4.19, +1.00 and −4.66 F1. Section 5.4 accounts for the HotpotQA reversal.

Small samples lacked power rather than validity. At $n = 50$ the IterCOMP-X−Raw gaps fell inside the paired detection threshold (18.0 F1) on both datasets, so we reported them as inconclusive and held the accuracy claim until the full 1,003-question VimQA run resolved it above; the underlying power analysis is released with the results. An early reading that the compression margin grows with task complexity likewise reverses at proper samples (+15.98 on VimQA at $n = 1{,}003$ against +10.72 on MuSiQue at $n = 1{,}500$), and does not hold at full scale.

5.2 Comparison with the original paper

Absolute numbers are not comparable across readers, but two invariants indicate a faithful reimplementation (Table 6). First, the **method ordering reproduces** - Oracle > IterCOMP > Raw > LLMLingua-2 on MuSiQue at both sample sizes, and on 2Wiki at scale (Section 5.1). Second, the Oracle/Raw ratio is in range: 1.74 at $n = 50$ and 2.12 at $n = 1{,}500$ against the paper's 1.88. That ratio is the headroom perfect selection buys over full context; being a property of the task and data loader rather than the reader, its agreement evidences that decomposition, gold-sentence labels and F1 are all correct.

Matching the operating point. A direct $n = 500$ comparison on MuSiQue puts IterCOMP 0.62 F1 *below* the paper's (26.74 against 27.36), while Oracle and Raw - which our compression code never touches - sit far above it (+12.33 and +3.51), our stronger reader alone. That −0.62 is misleading: it compares systems

Table 6. F1 from the original paper (LLaMA-3-8B reader) against our reproduction (Qwen2.5-7B at 4-bit, $n = 50$), plus our Vietnamese results. VimQA has no published baseline; the paper does not evaluate it.

Method	MuSiQue		VimQA
	paper	ours	ours
Oracle	37.51	57.0	53.3
Raw Documents	19.92	32.8	43.8
LLMLingua-2	14.72	13.5	30.7
IterCOMP	**27.36**	**35.1**	**43.0**
ratio	*0.14*	*0.32*	*0.19*
Oracle/Raw	*1.88*	*1.74*	*1.22*

at different operating points, 0.349 retention against 0.14. Sweeping k to match compression (released with the results) puts $k = 96$ at 0.160 retention and 29.04 F1, now +1.68 over the paper's 27.36. A residual gap remains - a reader +12.33 F1 stronger on perfect evidence should lift a working compressor by more than 1.68 - but it is within the noise floor, not the deficit the mismatched comparison implied.

5.3 What the compression costs to run

The saving is in tokens read, not in time. Two extra LLM calls per pass over 3.1 passes raise wall-clock per question even as the context shrinks: 2.88 to 4.52 s on VimQA and 2.65 to 8.73 s on MuSiQue, against contexts cut to 21% and 34% of Raw; 2Wiki and HotpotQA behave alike. LLMLingua-2, a single pass, is faster than Raw everywhere.

Which matters depends on where the model runs. Behind a per-token API, reading a fifth of the context is the bill and the extra passes are the vendor's problem. Self-hosted the arithmetic inverts: the GPU is the budget, and tripling latency to save context is a poor trade unless the context would not have fitted.

5.4 The ordering is reader-dependent

Oracle exceeds Raw on every dataset while reading 5.7% to 16.4% of the tokens, confirming the premise the literature rests on: irrelevant passages *do* degrade accuracy. But the size of that gap decides whether compression can exploit it. Oracle/Raw is 2.12 on MuSiQue and 1.30 on VimQA, where IterCOMP holds its own, but only 1.21 on HotpotQA, where Raw wins outright. **The premise is reader-dependent**: a reader that already copes with distractors leaves compression little to recover. Swapping our 7B reader for Qwen2.5-0.5B on HotpotQA, the compression pipeline fixed, restores IterCOMP to +7.66 over Raw - reversing the −4.66 at 7B, and recovering the ordering the original paper reports with its own weaker reader. The HotpotQA reversal is thus a property of the reader, not a defect of the compressor.

5.5 Vietnamese-specific findings

Running the method on Vietnamese surfaces four results. Boolean answer normalisation is worth +8.67 F1 on VimQA at $n = 1{,}003$, interval $[+6.98, +10.47]$, and exactly 0.0 on each of MuSiQue, 2Wiki and HotpotQA, isolating a language-specific fix from a metric-inflating one: a reader asked a Vietnamese question often answers "Yes", which is correct and scores 0 against the gold "đúng". The step is post-processing on the reader's output, gated by the *question*: it fires when the question ends in *phải không*, *đúng không*, *hay không* or the *có ... không* frame, never on the gold answer, so it is available at inference. It then reads the prediction's polarity against two closed lexicons - affirmative {yes, true, correct, right, đúng, có, phải, vâng, ừ} and negative {no, false, incorrect, wrong, không, sai, chưa} - and rewrites an unambiguous hit to the label of that polarity in the dataset's own vocabulary. A prediction matching neither, or both, is left untouched; one of the wrong polarity is rewritten to the opposite label and still scores zero, so the step cannot convert a wrong answer into a right one. Nominal compression ratios understate token cost on Vietnamese by 2.39× because the tokeniser that bills is not the one that counts; we propose an *effective ratio* that folds fertility back in. Vietnamese does not handicap the compressor: IterCOMP-X recovers 84% of the Oracle ceiling on VimQA (51.29 against 61.03) but only 56% on MuSiQue (25.35 against 44.90). The gap tracks distractor density rather than language: VimQA offers ten candidate passages per question and MuSiQue twenty, so perfect selection is worth more on MuSiQue and a compressor has further to fall short of it.

Undiacriticised Vietnamese. Vietnamese is frequently typed without tone marks online, a setting no compression paper evaluates. Stripping diacritics is expensive: *má*, *mà* and *mã* collapse into one string, 20.8% of VimQA's distinct syllables merge, and both branches lose about 20 F1 (Table 7). What it does not do is change the ordering: gold leads by 4.81 with tone marks and by 3.99 and 4.05 without, no interval separating. The damage lands on the reader rather than the compressor - of the matched 20-point drop, 0.81 F1 is the compressor's.

An earlier version of this experiment showed the opposite. Stripping diacritics from the context also stripped the document titles while the supporting-fact annotations kept theirs, so the two stopped matching and the gold branch received an empty context on 48 of 200 questions. Stripping both sides recovers 8.4 and 9.9 F1 for gold, and the effect disappears.

6 Error Analysis

We hand-categorised 50 IterCOMP predictions on VimQA (Table 8). Most errors belong to the *reader*: Oracle sees the gold evidence yet still reaches only 61.03 F1. Of IterCOMP's 48.7 F1 of error, 39.0 - **80%** - survives even perfect retrieval, so no compressor can address it.

With the 7B reader, **10%** of errors are not compression failures at all: yes/no questions answered correctly in substance but in the wrong surface form (F1 of

Table 7. Undiacriticised Vietnamese ($n = 200$, 7B reader). Gold context leads in every setting and none of the three gaps separates: the paired bootstrap intervals are $[-10.93, +1.44]$, $[-9.51, +1.50]$ and $[-9.48, +1.48]$. Stripping tone marks costs both branches about 20 F1, and costs them equally. `strip` leaves the gold answer accented, so the reader must restore marks it never saw; `both` strips the reference too, scoring entity recovery rather than diacritics.

Setting	IterCOMP-X	Gold	Δ
`keep`	50.17	**54.97**	−4.81
`strip`	29.91	**33.90**	−3.99
`both`	30.25	**34.30**	−4.05

Table 8. IterCOMP error distribution on VimQA ($n = 50$) under two readers. Format errors fall from 32% to 10% with the larger reader, a reader limitation rather than a compression failure.

Category	0.5B	7B
Fully correct (F1 = 1)	8%	16%
Partially correct (0 < F1 < 1)	34%	32%
Format error on yes/no	**32%**	**10%**
Fully wrong (F1 = 0)	26%	42%

0). That bucket was **32%** under a 0.5B reader - "tức là yes" for "đúng", a small model code-switching into English - and falls on the identical prompt as the reader grows, so boolean answers need language-aware normalisation, released with the code, rather than a compression fix. Partially correct answers give a related but wrong entity within a correctly retained segment; fully wrong ones are mostly compositional, where the Raw-Oracle gap is widest, so distractors rather than the compressor drive them.

A further **9.8% of VimQA is a single question template** - "is X older than Y?" - on which 44.9% of answers score zero against 37.1% elsewhere, because the task is arithmetic rather than retrieval. Under a 0.5B reader the same template collapses to 92.1% zeros.

7 Limitations and Ethical Considerations

Limitations. Absolute numbers are not comparable to the paper's: no prompts were released (ours are released with the code), the reader differs (Qwen2.5-7B at 4-bit vs. LLaMA-3-8B), and our retention is 0.32 at $n = 50$ and 0.349 at $n = 500$ against their 0.14. Sample sizes are uneven but no longer thin: all four datasets run at $n \geq 1{,}003$, the ablations at $n = 500$, the baselines at $n = 200$; what remains at $n = 50$ is the scorer ablation, reported as inconclusive. One baseline is missing, R2C releasing no public checkpoint; every other baseline with a released model is included, CompAct among them. The domain is narrow, since VimQA derives from Wikipedia. We do evaluate *undiacriticised* Vietnamese (Table 7), at

$n = 200$, where the intervals are wide enough that the setting bounds the damage rather than ranking the methods. Fertility is measured on translated text [3], so it holds for the tokenisers we bill against rather than the language at large. Every headline number comes from one reader, Qwen2.5-7B at 4-bit; decoding is greedy, so nothing here is sampling noise, but a quantisation artefact would not show up as variance either. The 0.5B run in Section 5.4 is the check we have: it moves the HotpotQA gap the way the reader-dependence account predicts. A 3B run we do not use tells against itself - its Oracle falls below its Raw, which cannot be right - and is released so that the reader can see it. The sharpest limit is that fertility is measured on four writing systems while compression is run on two, so the effective ratio is argued generally and shown on one language pair. Hindi, the worst case at 4.97×, is where it would matter most and is what we could not test: no Hindi or Thai multi-hop benchmark carries the sentence-level supporting facts the Oracle branch needs.

Ethics. VimQA's CC BY-NC-SA 4.0 licence confines us to research use, and because compression is lossy - it can drop a negation with no loss of fluency, silently inverting an answer - sensitive deployments should verify outputs against the source; the fix worth 8.67 F1 in Vietnamese yet 0.0 in English is a further caution that "multilingual" models are not equally reliable across languages.

8 Conclusion

We reproduced IterCOMP without its code, gave it its first Vietnamese evaluation, and release the configuration as IterCOMP-X. The central claim holds - sentence-level beats token-level compression on all four datasets - but the full ordering is reader-dependent: a stronger reader gains enough from full context that compression's accuracy advantage narrows or reverses. Vietnamese itself does not handicap the compressor; what we could not move is the reader, failing on ∼30% of questions even given gold evidence.

Acknowledgments. This research is supported by research funding from the Faculty of Information Technology, University of Science, Vietnam National University - Ho Chi Minh City.

References

1. Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N.A., Tsvetkov, Y.: Do all languages cost the same? tokenization in the era of commercial language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.614/
2. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.137/

3. Deutsch, D., Briakou, E., Caswell, I., Finkelstein, M., Galilee, R., Juraska, J., Kovacs, G., Lee, A., Rudder, J., Freitag, M.: WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In: Proceedings of WMT (2025), https://huggingface.co/datasets/google/wmt24pp
4. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.825/
5. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In: Proceedings of ACL (2024), https://aclanthology.org/2024.acl-long.91/
6. Le, K., Nguyen, H., Le Thanh, T., Nguyen, M.: VIMQA: A vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In: Proceedings of LREC (2022), https://aclanthology.org/2022.lrec-1.700/
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of NeurIPS (2020)
8. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.391/
9. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for vietnamese. In: Findings of EMNLP (2020), https://aclanthology.org/2020.findings-emnlp.92/
10. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.Y., Zhao, H.V., Qiu, L., Zhang, D.: LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.57/
11. Petrov, A., La Malfa, E., Torr, P., Bibi, A.: Language model tokenizers introduce unfairness between languages. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
12. Phan, L., Tran, H., Nguyen, H., Trinh, T.H.: ViT5: Pretrained text-to-text transformer for vietnamese language generation. In: Proceedings of NAACL Student Research Workshop (2022), https://aclanthology.org/2022.naacl-srw.18/
13. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP (2023), https://aclanthology.org/2023.findings-emnlp.378/
14. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL (2023), https://aclanthology.org/2023.acl-long.557/
15. Xu, F., Shi, W., Choi, E.: RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of ICLR (2024), https://openreview.net/forum?id=mlJLVigNHp
16. Yun, J., Kim, Y.: IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In: Proceedings of ACL (Volume 1: Long Papers). San Diego, California (2026), https://aclanthology.org/2026.acl-long.1559/